Prahari — Presentation & Project Guide

Slide-by-slide script, every diagram explained simply, and the whole project in plain words

FinSpark'26 · Bank of Maharashtra · Problem Statement 1 — Privileged Access Misuse & Insider Thr

Repository: github.com/positromen/FinSpark-26-Trial

Contents

How to read this guide

This document has five parts. Read them in order the first time; after that, jump to whatever you need.

1. **The whole project in plain words** — what Prahari is, told like a story, with everyday examples.
2. **The 16 slides** — for *each* slide: what it is for, what you **must** put on it, what you **could** add, and a simple script of **what to say** out loud.
3. **Every diagram explained simply** — all eight pictures, walked through box by box.
4. **Every word explained simply** — a plain-English dictionary of every technical term we use.
5. **Cheat-sheet** — the exact numbers, names, passwords and facts to memorise.

Throughout, anything you can literally read aloud in the demo is written in *plain, short sentences.*

PART 1 — The whole project in plain words

The one-sentence version

Prahari watches the most powerful people inside a bank, notices in real time when one of them starts doing something dangerous, automatically stops them, and keeps the proof safe from future super-computers.

The problem, told simply

A bank protects itself from *outsiders* — hackers on the internet — with firewalls and antivirus. That is like locking the front door.

But some people are already **inside** the bank with keys: the database admins, the IT staff, and outside **vendors/contractors**. These are called **privileged users** — think of them as staff who hold the **master key** to every locker. If one of them turns bad, is careless, or has their account stolen, the front-door lock does not help at all. The danger is already inside.

The competition (Bank of Maharashtra, FinSpark'26) asked us to catch this **insider** danger. It says the insider can be one of **three kinds**:

- **Malicious** — a bad person on purpose. Example: a contractor whose contract ended, but whose account was never switched off, quietly logs in at 2 in the morning and copies 5,000 customers' details to steal them.
- **Negligent** — a good person being careless. Example: a vendor whose access *expired* two weeks ago is still logging in from their **personal laptop** and opening customer files. No bad intent — but a serious rule-break.
- **Compromised** — a good person whose account was **stolen** by a hacker. Example: a normal employee's login suddenly appears from another country, on a device never seen before, doing things inhumanly fast.

On top of that, the bank must keep its **secrets and its records safe even from future quantum computers** — because a criminal can steal encrypted data today and unlock it years later once quantum computers are strong enough. This is called **"harvest now, decrypt later."**

Our solution, told simply

Prahari (a Hindi word meaning **"sentinel / guard"**) sits *between* the privileged staff and the important bank systems and does **four things** for every session (a session = one login-to-logout period):

1. **Watches and records.** Every action (a database query, a file open, a config change, a bulk export) is written down like a CCTV recording — you can replay it later, command by command.
2. **Scores the risk, live.** Every action gets a **risk score from 0 to 100**, calculated instantly by two "brains" working together: a **rule engine** (a checklist of known-bad behaviours) and an **AI model** (learns what "normal" looks like and flags the unusual). It always explains *why* in plain English.
3. **Responds automatically.** Based on the score, Prahari either **allows** the action, asks for **extra verification (step-up MFA)**, **holds it for a second manager's approval (maker-checker)**, or **blocks it outright** and locks the account.
4. **Protects the proof.** The bank's stored passwords go in a **quantum-safe vault**, and every record in the audit log is **chained and digitally signed** so that if anyone edits even one letter, the system instantly proves the tampering.

What we actually built (it's real, not slides)

It is a working **two-sided web app**:

- The **Employee Portal** — what a bank staff member sees. It is a **real banking desk**: they log in, open accounts, move money, and clear approvals — and every action is scored and enforced *live*. A big transfer is held for a second officer, and a suspicious one is flagged and **flashes a red banner** on the screen.
- The **SOC Console** — what the **Security Operations Centre** analyst sees. A live control room: which sessions are active, their risk, alerts, the AI's reasoning (an **AI Model Insights** panel that shows *which behaviours* moved the score), one-click **Lock / Approve / Dismiss** buttons, an impact-metrics strip, the session replay, a risk heatmap, the access review, and the quantum-safe audit log.

You can run it on **two computers at once** on the same Wi-Fi: an employee acts on one, and the security team watches it happen **live** on the other, with no refreshing. And it starts on **any computer with no compiler or special setup** — the quantum-safe crypto has a built-in pure-Python fallback, so a teammate never hits a build error.

The six things the competition asked for — all done

1. **Detect misuse of privileged accounts** — done (rule engine).
2. **Identify insider threats in real time** — done (live scoring + WebSocket push).
3. **AI-driven behavioural analysis** — done (IsolationForest + per-user baselines).
4. **Risk-based access control** — done (allow / MFA / maker-checker / block).
5. **Protect critical administrative systems** — done (blocks the action before damage; locks the account).
6. **Quantum-proof cryptography** — done (ML-KEM-768 vault + ML-DSA-65 signed audit log).

PART 2 — The 16 slides

The official template has **16 slides**. Below, each slide has four boxes:

- GOAL — **What it is for** — the template's own instruction.
- NEED — **Must include** — put this on the slide.
- OPTIONAL — **Could include** — add if you have room or the judges probe deeper.
- SCRIPT — **Say this** — a short script you can read almost word-for-word.

Keep slides light (few words, one picture). The detail lives in your *mouth* (the script) and in this guide, not on the slide.

Slide 1 — Title

GOAL — *Team name, team bio, date.*

NEED — Must include:

- Project name: **Prahari** — with the tagline **"Privileged-Access Insider-Threat Detection & Management for Banks."**
- Team name, each member's name, the date.
- Problem statement line: **PS1 — Privileged Access Misuse & Insider Threat Detection.**

OPTIONAL — Could include: the GitHub link (`github.com/positromen/FinSpark-26-Trial`); a one-line hook: *"Catch a risky insider in real time — and keep the proof quantum-safe."*

SCRIPT — Say this: > "Good morning. We are team [name]. Our project is **Prahari** — the Hindi word for *sentinel*. It solves Problem Statement 1: catching **privileged-access misuse and insider threats** inside a bank, in real time, and protecting the evidence with quantum-safe cryptography."

Slide 2 — Problem Statement

GOAL — *State the problem you chose and why.*

NEED — Must include:

- Banks defend well against **outsiders**, but the costliest breaches come from **trusted insiders** who already hold privileged access.
- The three insider types: **malicious, negligent, compromised** — with a one-line example of each.
- The "harvest now, decrypt later" quantum risk to stored secrets and audit records.

OPTIONAL — Could include: dormant/vendor accounts never de-provisioned; tamperable audit logs; the RBI compliance angle.

SCRIPT — Say this: > "A bank's firewall stops outsiders, but it is blind to the DBA, the sysadmin, or the vendor who already has the master key. The problem statement names three faces of this insider: **malicious** — a dormant vendor account that wakes at 2 AM and bulk-exports 5,000 customer records; **negligent** — a vendor whose access expired but who still pulls files from a personal laptop; and **compromised** — a normal account suddenly logging in from another country on a new device. We chose this because it is the highest-impact risk for a bank, and it let us combine three hard things — real-time AI, access control, and post-quantum cryptography — into one solution."

Slide 3 — Pre-Requisite

GOAL — *Assumptions, required access, data, tools, environments, inputs.*

NEED — Must include:

- **Assumption:** privileged activity is available as a stream of events. For the demo, a built-in **simulator** produces it; in production it comes from the bank's **PAM/SIEM** logs.
- **Inputs per event:** who (user), what (action), which system (resource), how much data (records touched), from where (IP/location), on what (device), and when (timestamp).
- **Environment:** Python 3.11+, a browser; runs **fully offline** on SQLite. One-time internet only to `pip install` — **no compiler / build step** (the quantum library has a pure-Python fallback).

OPTIONAL — Could include: access to the IAM directory (roles, vendor/dormant flags, access-expiry dates); no secrets stored in the repo; `.env`-based config.

SCRIPT — Say this: > "Our only assumption is that privileged activity arrives as events — user, action, system, data volume, location, device, time. In a real bank that feed comes from existing PAM and SIEM tools; for the demo we generate the same shape with a simulator. It runs completely offline on SQLite — important for an unreliable venue Wi-Fi."

Slide 4 — Tools or Resources

GOAL — *Technologies, frameworks, libraries used.*

NEED — Must include (a simple table works):

- **Backend:** Python, **FastAPI** (web server), **SQLAlchemy** (database), **SQLite** (storage, swappable to PostgreSQL).
- **AI: scikit-learn IsolationForest** + per-user behavioural baselines.
- **Post-quantum crypto: ML-KEM-768** (FIPS 203) and **ML-DSA-65** (FIPS 204); **AES-256-GCM** for the payload. Native **liboqs** when installed, else a **pure-Python** provider (`kyber-py` / `dilithium-py`) — no compiler.
- **Banking:** a real ledger (accounts + transactions) with maker-checker and fraud controls.
- **Real-time: WebSocket**.
- **Frontend: React + Vite + Tailwind**.
- **Testing: pytest — 40 automated tests.**

OPTIONAL — Could include: PBKDF2 password hashing + signed tokens; Docker; one-command run script.

SCRIPT — Say this: > "Standard, production-grade tools. Python and FastAPI for the backend, scikit-learn's IsolationForest for the AI, React for the two dashboards, a real banking ledger under the portal, and — the special part — real NIST post-quantum algorithms, which fall back to a pure-Python build so they run on any laptop with no compiler. Everything is covered by 40 automated tests."

Slide 5 — Supporting Functional Documents

GOAL — *User flows, process notes, logic flows, references.*

NEED — Must include: name the documents you ship in the repo —

- **README** (quick start + demo accounts),
- **DOCUMENTATION** (full technical doc with diagrams),
- **DEMO_SCRIPT** (click-by-click narration),
- this **Presentation & Project Guide**.
- Two **user flows**: Employee Portal (login -> act -> allow/MFA/hold/block) and SOC Console (watch -> investigate -> respond).

OPTIONAL — Could include: the logic-flow diagrams (detection -> score -> response, the vault, the audit chain).

SCRIPT — Say this: > "Everything is documented in the repo — a README, a full technical document with all the diagrams, a click-by-click demo script, and a presentation guide. We also map two user journeys: the employee doing their job, and the analyst watching and responding."

Slide 6 — Key Differentiators & Adoption Plan

GOAL — *What makes you different, and how a bank would adopt it.*

NEED — Must include — **Differentiators:**

- We handle **all three insider types**, each with a **different, correct response** — most tools only do anomaly detection.
- **Explainable AI** — every score comes with plain-English reasons and per-user factors, not a black box.
- **Real PAM features** — session **recording/replay** and an **access review** — plus **live enforcement** that blocks the action mid-way.
- **A real banking desk, not a mock** — staff actually move money; a high-value transfer is **held** for a second officer and a suspicious one is **flagged** and flashed, so insider risk is shown on real transactions.
- **Working post-quantum** vault and tamper-evident audit log — with a **pure-Python fallback so it runs on any machine with no compiler**.

Adoption plan (low risk -> high):

1. **Monitor mode** — deploy read-only beside existing tools; learn baselines; no blocking.
2. **Assist mode** — turn on step-up MFA and maker-checker for medium risk.
3. **Enforce mode** — auto-block clear attacks; move crown-jewel credentials into the quantum-safe vault.

OPTIONAL — Could include: integrations (CyberArk/BeyondTrust PAM, Splunk/QRadar SIEM, the bank's SSO).

SCRIPT — Say this: > "Three differentiators. One: we cover all three insider types with distinct responses. Two: the AI is fully explainable. Three: we ship real post-quantum crypto, not a promise. Adoption is staged — start in monitor-only mode to build trust, then switch on step-up verification, and finally auto-blocking."

Slide 7 — GitHub Repository Link & supporting diagrams

GOAL — *Repo link + diagrams/screenshots.*

NEED — Must include:

- `github.com/positromen/FinSpark-26-Trial`
- The **architecture diagram** (see Part 3, Diagram 1).
- A note: **one-command offline run; 40 tests; prebuilt UI committed.**

OPTIONAL — Could include: the demo accounts table; a QR code to the repo.

SCRIPT — Say this: > "Here is the repository. It runs with a single command, fully offline, and ships with 40 passing tests and the prebuilt UI. This is our architecture at a glance — I'll walk through it properly on the architecture slide."

Slide 8 — Business Potential and Relevance

GOAL — *Future potential, real-world use, long-term value.*

NEED — Must include:

- Insider incidents are among the **costliest** breach types; one misused privileged account can expose **millions** of customer records and trigger regulatory penalties.
- Directly supports **RBI** expectations: least privilege, immutable audit trails, privileged-access oversight.
- **Quantum-readiness now** aligns with global migration timelines (NIST FIPS 203/204).

OPTIONAL — Could include: expansion — transaction-fraud correlation, just-in-time access, workforce-wide insider-risk scoring; deployable as an internal platform **or** SOC-as-a-service.

SCRIPT — Say this: > "Every bank with privileged users needs this. A single misused admin account can leak millions of records and draw regulatory fines. Prahari maps directly to RBI's governance expectations, and its post-quantum layer future-proofs the bank against 'harvest now, decrypt later'. It extends naturally to transaction-fraud correlation and just-in-time access."

Slide 9 — Uniqueness of Approach and Solution

GOAL — *What is innovative or original.*

NEED — Must include:

- **One engine, three insider types** — and a **type-aware policy**: negligence is sent to a **human review**, never auto-blocked like an attack.
- **Rules + AI fused into one explainable 0–100 score** with a per-user "unusual for *this* person" layer.
- **PAM + UEBA + post-quantum in one offline-capable product** — a combination competitors don't ship together.

- **Live enforcement** — it stops the action *in progress* and **locks the account**; a blocked user cannot simply log in again.
- **Tamper-evident, quantum-safe audit** — you literally cannot silently edit the log.

SCRIPT — Say this: > "The novelty is the combination. One engine classifies the insider type and responds appropriately — we never hard-block mere negligence. The AI explains itself per user. And we put PAM, behavioural AI, and NIST post-quantum crypto into a single product that runs offline — which nobody else ships together."

Slide 10 — User Experience

GOAL — *How users interact; why it is simple.*

NEED — Must include:

- **Two role-based experiences, one login.**
- **Employee Portal:** a real banking desk (**Accounts, Payments, Transactions, Approvals**) plus the action console; feedback is instant and obvious — green *allowed*, an amber **MFA pop-up**, an orange **held-for-approval**, a red **flagged/fraud flash banner**, or a full-screen red **BLOCKED**.
- **SOC Console:** one screen of situational awareness — live sessions, risk gauge, typed alerts, the AI's reasoning, timeline, session replay, heatmap, access review, and one-click **Lock / Approve / Dismiss** with confirmation.
- Colour-coded insider types; new alerts **flash**; everything is explained in plain words.

OPTIONAL — Could include: dark, control-room SOC theme; keyboard-friendly; a live **LIVE** connection indicator.

SCRIPT — Say this: > "Two audiences, one app. The employee just does their job and gets immediate, self-explanatory feedback. The analyst sees a single control-room screen — every alert is colour-coded by insider type and written in plain English, so triage takes seconds. And the analyst can lock, approve, or dismiss a session in one click, with a confirmation that it's sealed to the audit chain."

Slide 11 — Scalability

GOAL — *Scale across branches, users, systems, volumes.*

NEED — Must include:

- **Stateless scoring** — add more servers behind a load balancer to handle more load.
- **SQLite -> PostgreSQL** by changing one line (pure ORM).
- **WebSocket fan-out** via a Redis/Kafka broker for many SOC screens and high event rates.
- Ingests real **PAM/SIEM** feeds — millions of events per day — on the same event schema.
- The AI **retrains per role/shift**; the scoring is **sub-second**.

SCRIPT — Say this: > "It scales three ways. The scoring service is stateless, so you just add servers. Storage moves from SQLite to Postgres with one connection string. And it plugs into existing PAM/SIEM feeds at bank scale — millions of events a day — because everything maps to the same event schema."

Slide 12 — Ease of Deployment and Maintenance

GOAL — *How practical to implement, run, maintain.*

NEED — Must include:

- **One command** (`run.ps1` / `run.sh`) sets up everything and runs **fully offline**.
- **No compiler, no build step** — the post-quantum layer auto-falls-back to pure Python, so it starts on any teammate's machine (no *"oqs shared libraries not found"* wall).
- **Two-computer LAN demo** — the server binds `0.0.0.0`, so a second machine watches live at `http://<host-ip>:8000`.

- **Docker** ready.
- Config via `.env`; **no secrets in the repo**.
- Small, modular, **type-hinted** code; **40 automated tests** guard every change.
- Post-quantum crypto sits behind **one swappable module** (native library *or* pure Python).

SCRIPT — Say this: > "One command brings up the database, the AI, both dashboards and the API — offline, with no compiler needed, so it runs on any laptop. It's containerised, fully tested, and the crypto is behind a single interface with a pure-Python fallback, so providers swap without touching the app. A new engineer can run it in under a minute, and two computers can share one live demo over Wi-Fi."

Slide 13 — Security Considerations

GOAL — *Cyber risks, data protection, access control, compliance.*

NEED — Must include:

- **Data minimisation** — we score on **activity metadata**, not customer money data / PII.
- **Credential protection** — the vault seals secrets with **ML-KEM-768 + AES-256-GCM** (quantum-safe).
- **Audit integrity** — every entry is **hash-chained and ML-DSA-65 signed**; tampering is provable.
- **Enforcement is server-side** — blocked accounts stay locked; a challenged action isn't saved until it's actually allowed (no gaming the score).
- **Access control** — passwords are **PBKDF2-hashed**, sessions use **signed tokens**, and SOC APIs are **analyst-only**.
- **Offline by design** — no external calls at run time; keys never committed.

SCRIPT — Say this: > "Security is the point of the product, so it's security-first internally too. We only look at activity metadata, never customer PII. Credentials go in a quantum-safe vault. The audit log is hash-chained and post-quantum signed, so tampering is mathematically provable. Enforcement is server-side — you can't bypass it from the browser — and the SOC endpoints are locked to analysts only."

Slide 14 — Architecture Diagram & Images

GOAL — *A clear architecture diagram.*

NEED — Must include: the **architecture picture** (Part 3, Diagram 1) and a one-line caption. Optionally the **scoring** and **post-quantum** diagrams.

SCRIPT — Say this (point at the boxes left to right): > "Privileged activity comes in on the left and is normalised into sessions. It fans out to two detectors — the behavioural AI and the rule engine — which feed one scoring engine that produces a 0-to-100 score with reasons. The score drives the response — allow, MFA, maker-checker, or block — and lights up the SOC console. And wrapping the whole thing is the post-quantum security layer: the ML-KEM vault and the ML-DSA-signed audit log."

Slide 15 — Solution Screenshots, Video & GitHub Link

GOAL — *Prototype screenshots, demo video, GitHub, proof it works.*

NEED — Must include (drop real screenshots):

- **Login** page.
- **Employee Portal** with the full-screen **BLOCKED** overlay.
- **SOC Console** — live blocked session + a flashing **CRITICAL** alert.
- **AI Model Insights** — feature bars + the risk-score trajectory.
- **Audit Chain** — green **VERIFIED**, then red **FAILED** after tamper.
- The **GitHub link** and the **demo video link**.

SCRIPT — Say this: > "These are real screenshots from the running product — not mock-ups. Here's the block landing live, the SOC alert firing, the AI explaining the score feature by feature, and the audit chain catching a tamper. The repository and a full demo video are linked."

Slide 16 — Thank You

GOAL — *Team names, roles, contacts, closing note.*

NEED — Must include: names, roles, one contact, and a closing line.

SCRIPT — Say this: > "That's Prahari — detect with explainable AI plus rules, respond in real time, and keep the proof quantum-safe. Thank you — we're happy to take questions or run the live demo."

PART 3 — Every diagram explained simply

We have **eight** diagrams. For each: what it is, a walk through every box, and why it matters. Imagine each picture is a little comic strip that reads left-to-right and top-to-bottom.

Diagram 1 — System Architecture (the big picture)

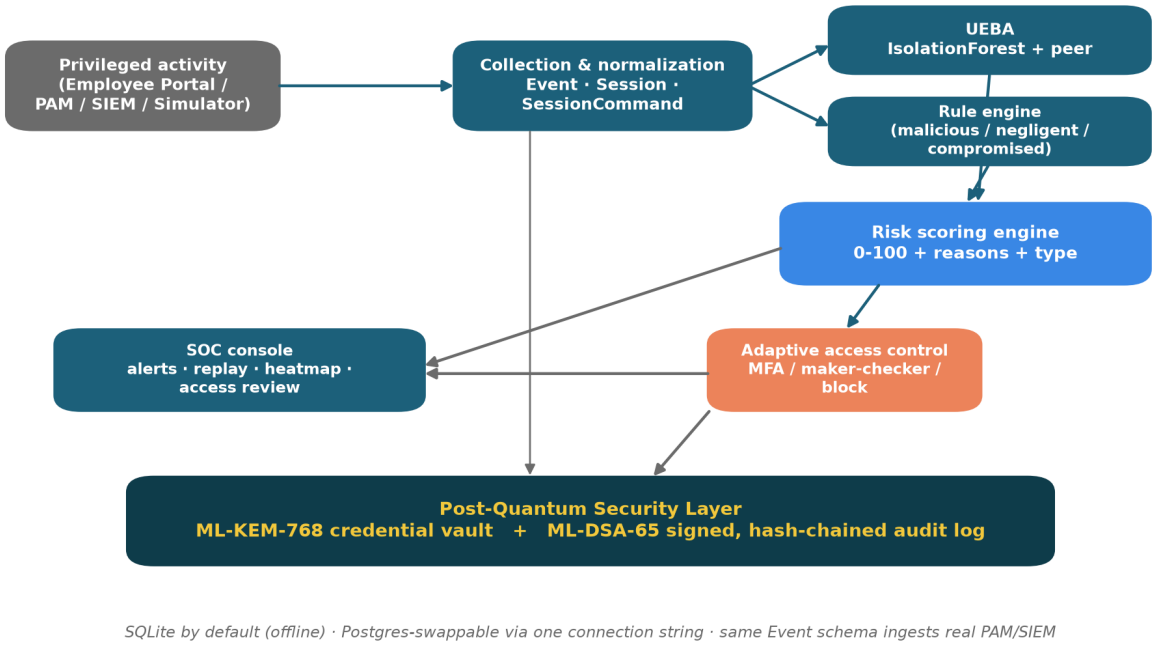

Figure 1: System architecture — six modules from ingestion to the post-quantum layer.

Think of a **factory conveyor belt** with six stations. A "raw material" (a privileged action) enters on the left and comes out the other side either allowed or blocked, with a permanent, tamper-proof receipt.

- **Box 1 — Privileged activity (grey, top-left).** Where actions come from: our Employee Portal, or in a real bank, the PAM/SIEM logs, or our simulator. *(The raw material.)*

- **Box 2 — Collection & normalization (teal).** Turns every action into a tidy, standard record — a **Session** with **Events** and a recorded **command trail**. *(Cleaning and labelling the raw material so every later station understands it.)*
- **Box 3 — UEBA (teal, top-right).** The **AI brain**: it learned what "normal" looks like and measures how *unusual* this session is. *(A station that says "hmm, this looks weird.")*
- **Box 4 — Rule engine (teal).** The **checklist brain**: fixed known-bad patterns (dormant account woke up, 5,000 records exported, out-of-role access…). *(A station with a printed list of forbidden moves.)*
- **Box 5 — Risk scoring engine (blue).** Combines the two brains into **one number 0–100**, plus the plain-English *why* and the insider *type*. *(The station that adds it all up.)*
- **Box 6a — Adaptive access control (orange).** Turns the number into an action: allow / step-up MFA / maker-checker / block. *(The gate that opens, half-opens, or slams shut.)*
- **Box 6b — SOC console (teal).** Shows everything to the human security team, live.
- **The wide dark bar at the bottom — Post-Quantum Security Layer.** Wraps everything: the **ML-KEM-768 vault** (locks secrets) and the **ML-DSA-65 signed, hash-chained audit log** (a receipt no one can forge). *(The tamper-proof printer under the whole belt.)*

Why it matters: it shows the judges that this is a *complete pipeline*, not a single trick — ingestion, two kinds of detection, scoring, response, a human console, and a crypto safety net.

Diagram 2 — Data Model (the database tables)

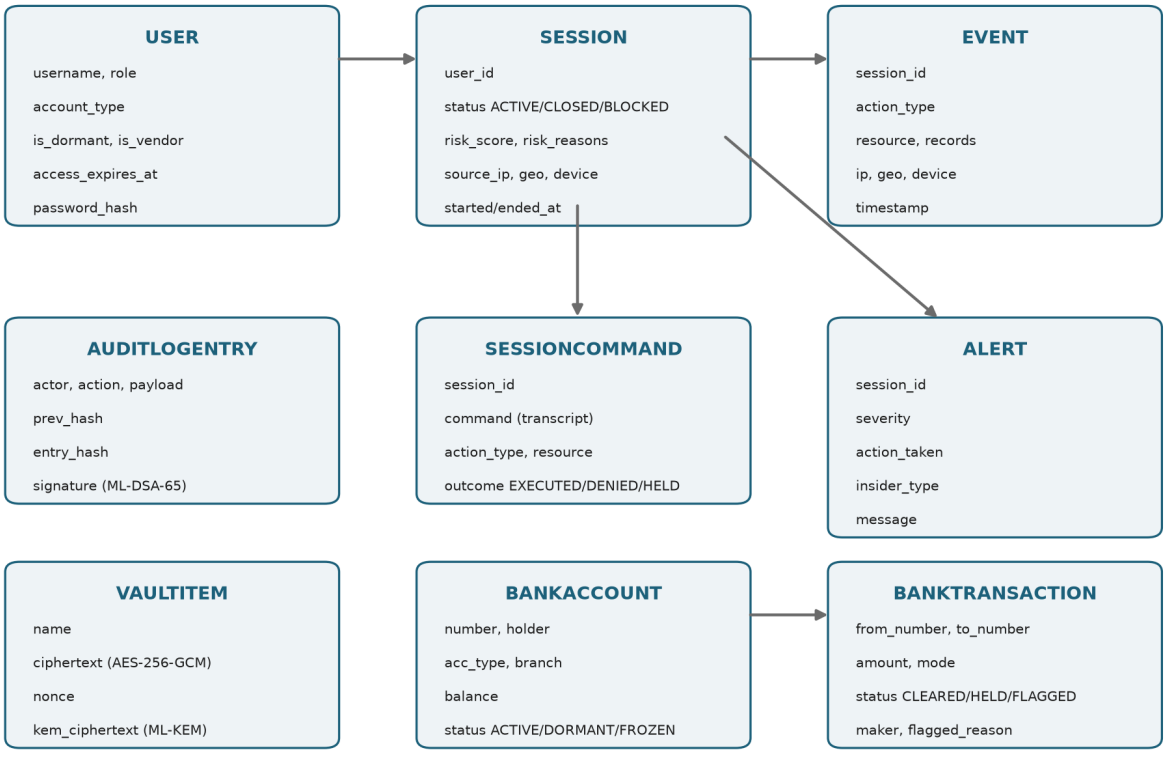

Figure 2: Entity model — the nine tables and how they connect.

This is the **filing-cabinet layout** — nine drawers and how they reference each other.

- **USER** — every privileged person: their name, role, whether they are dormant or a vendor, and when their access **expires**. *(The staff directory.)*
- **SESSION** — one login-to-logout window: its status (active/closed/**blocked**), its risk score, and where it came from (IP, location, device). *(One visit to the building.)*
- **EVENT** — one single action inside a session (a query, an export…), with how many records it touched and when. *(One thing the person did during the visit.)*
- **SESSIONCOMMAND** — the **recording**: the actual command line typed, with an outcome (executed / **denied** / held). *(The CCTV transcript of the visit.)*
- **ALERT** — a warning raised for a risky session: severity, the action taken, and the insider **type**. *(The note the guard writes.)*
- **AUDITLOGENTRY** — one tamper-proof record: who/what/when, the **hash of the previous entry**, and a **post-quantum signature**. *(A page in a sealed logbook where every page locks the one before it.)*
- **VAULTITEM** — one stored secret, encrypted; the key is sealed with post-quantum crypto. *(A locked box inside the safe.)*
- **BANKACCOUNT** — a real bank account: number, holder, type, branch, balance, and status (active/frozen). *(A customer's passbook.)*
- **BANKTRANSACTION** — one money movement: from/to, amount, mode, status (**cleared / held / flagged**), who made it, and any fraud reason. *(A line in the bank ledger.)*

The **arrows** mean "one-to-many": one USER has many SESSIONS; one SESSION has many EVENTS and a command recording; one BANKACCOUNT has many BANKTRANSACTIONS. **Why it matters:** it proves the data is properly structured and that "session recording," "access expiry," and a **real banking ledger** are first-class features — not just words.

Diagram 3 — Scoring Pipeline (how the number is made)

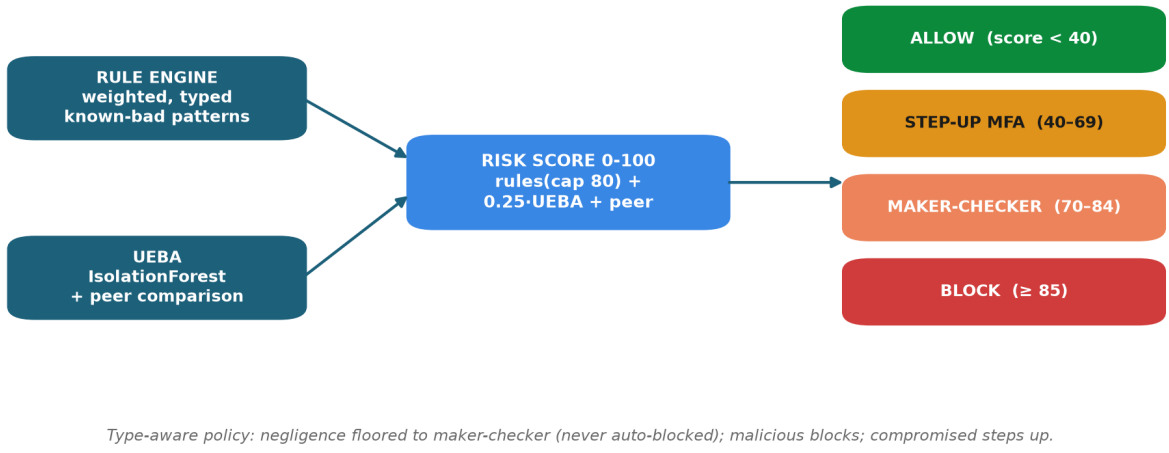

Figure 3: Scoring — rules and AI fuse into a 0–100 score that maps to the response ladder.

Read it **left to right**. Two inputs on the left flow into one score in the middle, which points at a **ladder** on the right.

- **Left, top — Rule engine.** Produces "rule points" (each broken rule adds weight), capped so rules alone can't dishonestly max the score.

- **Left, bottom — UEBA.** Produces the AI anomaly (0–100), which contributes up to **25 points** (it's a *nudge*, not the whole story).
- **Middle — the sum.** `score = rule points + 0.25 × AI anomaly + a peer-comparison bonus`, capped at 100.
- **Right — the response ladder** (four rungs, colour-coded):
  - **0–39 -> ALLOW** (green)
  - **40–69 -> STEP-UP MFA** (amber)
  - **70–84 -> MAKER-CHECKER** (orange)
  - **85–100 -> BLOCK** (red)

The caption line is the clever bit: **negligence is floored to maker-checker and never auto-blocked** — a careless employee is sent to a human, not slammed like an attacker.

Why it matters: it shows the AI and the rules *cooperate*, the score is *explainable*, and the response is *proportionate* to the risk.

Diagram 4 — The Three Insider Types (one engine, three responses)

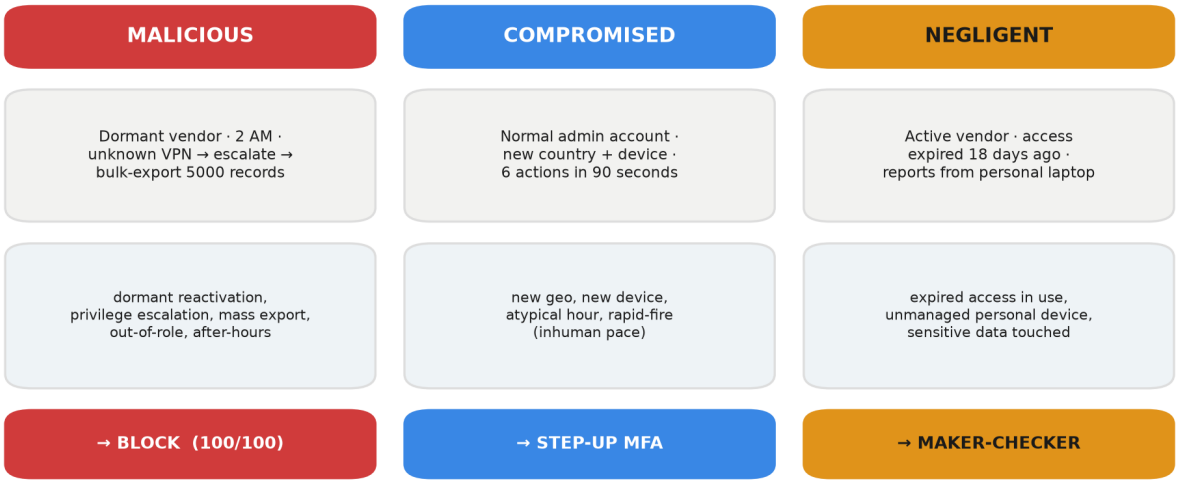

Figure 4: Three insider types — one engine, three distinct, correctly-typed responses.

Three columns, one per insider type. Each column reads top to bottom: **who they are -> the signals we detect -> the response**.

- **MALICIOUS (red).** Dormant vendor, 2 AM, unknown VPN -> escalates privilege -> tries to export 5,000 records. Signals: dormant reactivation, privilege escalation, mass export, out-of-role, after-hours. **Response: BLOCK (score 100).**
- **COMPROMISED (blue).** A normal admin account suddenly from a new country + new device, six actions in ninety seconds. Signals: new location, new device, atypical hour, inhuman speed. **Response: STEP-UP MFA.**
- **NEGLIGENT (amber).** An active vendor whose access expired 18 days ago, pulling reports from a personal laptop. Signals: expired access still in use, unmanaged personal device. **Response: MAKER-CHECKER review.**

Why it matters: the competition explicitly asks for all three. This one picture proves we handle each **distinctly and correctly** — the single most important slide for scoring points against the problem statement.

Diagram 5 — Live Enforcement (what happens the instant you click)

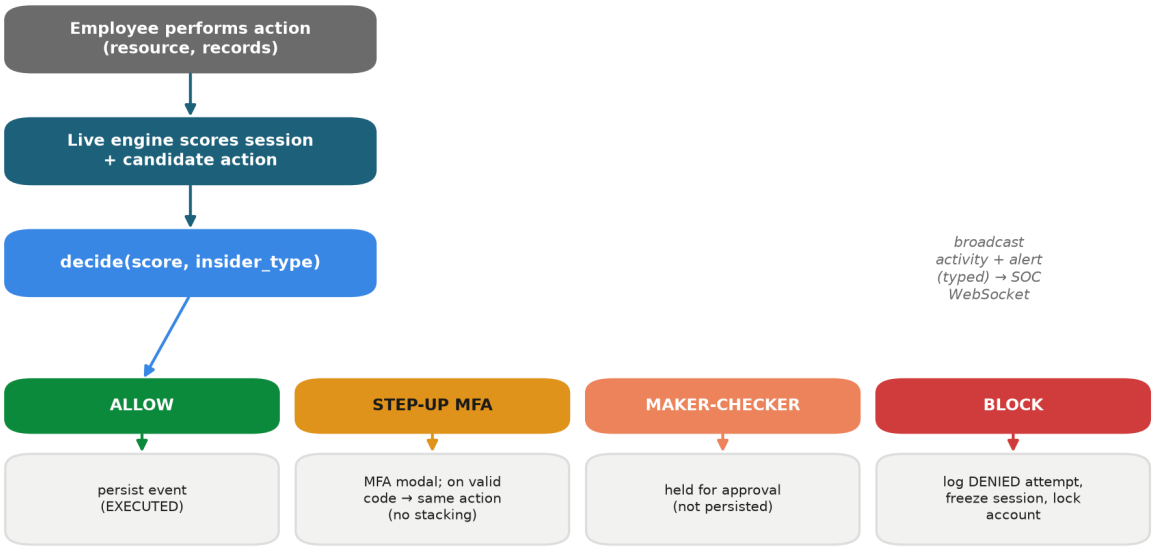

Figure 5: Live enforcement — every action is scored and routed to one of four responses before it takes effect.

A **flowchart of one action**, top to bottom.

1. **Employee performs an action** (chooses a system + record count).
2. **The live engine scores the whole session so far** *including* this new action.
3. `decide(score, insider_type)` picks the outcome, which branches four ways:
   - **ALLOW** -> the action is saved and completes.
   - **STEP-UP MFA** -> a code pop-up; enter the code and the *same* action proceeds (it is not re-counted — no score inflation).
   - **MAKER-CHECKER** -> held for a second approver (not saved yet).
   - **BLOCK** -> the attempt is logged as **DENIED**, the session is frozen, and the account is **locked**.
4. On the right, a note: **every action is broadcast to the SOC console over WebSocket**, so the security screen updates instantly.

Why it matters: it proves enforcement happens **before** the action takes effect (not after the damage), and that it's genuinely **live** across screens.

Diagram 6 — Post-Quantum Credential Vault

How a password is stored so that even a future quantum computer can't read it. Read left to right.

- **Secret** (e.g., a database root password) ->
- **AES-256-GCM encryption** locks it with a key that is ->
- the **shared secret produced by ML-KEM-768** (the post-quantum key-agreement, sealed against the vault's public key) ->
- **Stored:** the locked text + the sealed key. To read it back, you must run **ML-KEM decapsulate** with the vault's secret key, then decrypt.

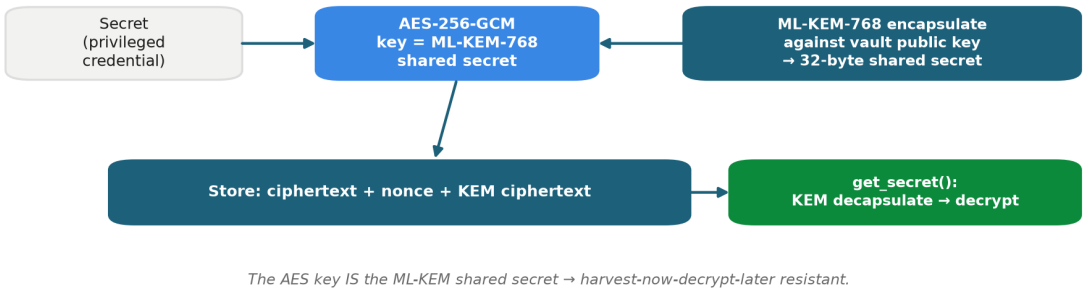

Figure 6: Credential vault — a secret is sealed with AES-256-GCM under an ML-KEM-768 shared secret.

The key insight (bottom caption): **the AES key *is* the ML-KEM shared secret.** So data copied today cannot be unlocked later — that defeats "harvest now, decrypt later."

Why it matters: it shows the post-quantum protection is *real and specific*, not a buzzword.

Diagram 7 — Tamper-Evident Audit Chain

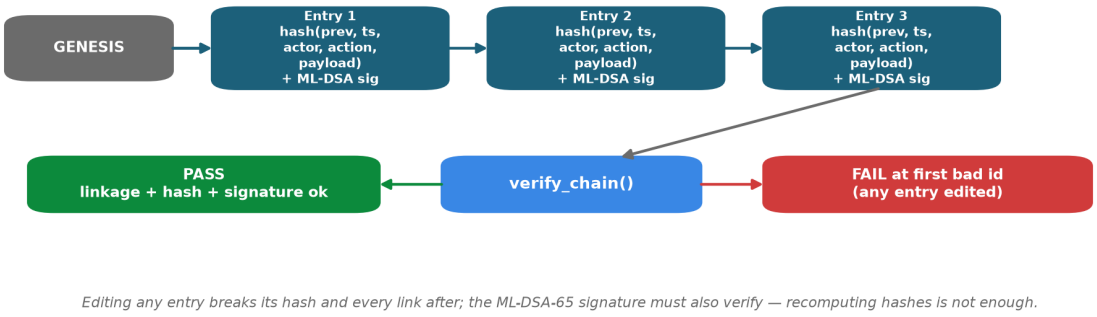

Figure 7: Audit chain — each entry hash-links the previous and is ML-DSA-65 signed; any edit fails verification.

How the logbook proves it was never edited. Read left to right.

- **GENESIS** -> **Entry 1** -> **Entry 2** -> **Entry 3** . . . each entry contains a **hash (fingerprint) of the previous entry** *and* its own **ML-DSA-65 digital signature**.
- On the right: **verify_chain()** checks every link and every signature.
  - If all good -> **PASS**.
  - If anyone edited even one entry -> the fingerprint no longer matches and the signature no longer verifies -> **FAIL, pointing at the exact broken entry.**

Because each page locks the page before it (like a blockchain) **and** each page is signed with a post-quantum signature, a tamperer would have to **forge a NIST post-quantum signature** — which is the whole point

of choosing ML-DSA.

Why it matters: in the live demo you click **Tamper**, and the chain instantly turns red at entry #1 — a dramatic, undeniable proof.

Diagram 8 — Core-Banking Transfer Flow (money meets the guard)

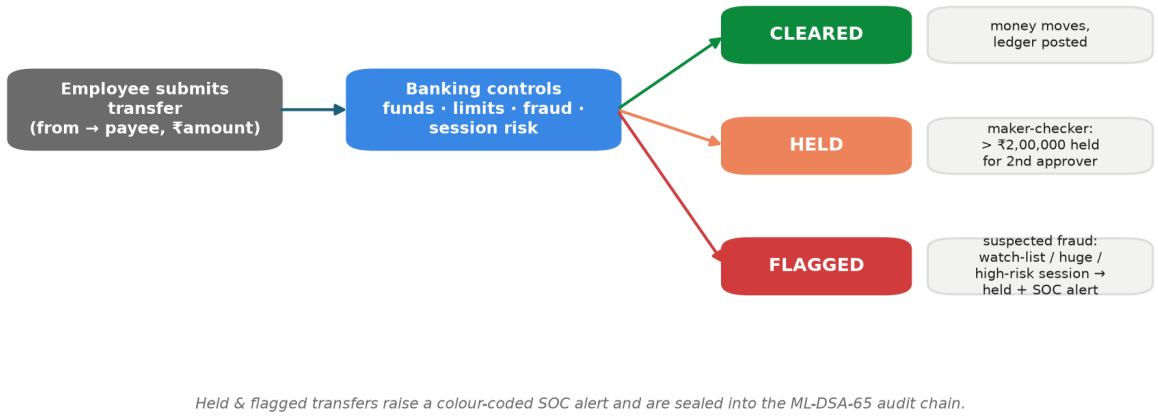

Figure 8: Core-banking flow — a transfer is checked for fraud and session risk, then cleared, held, or flagged.

This is what happens when a portal user clicks **Send** on a transfer. Read it top to bottom — one payment walks through three gates before any money moves.

1. **A transfer is submitted** (from account, to a payee, an amount, a mode like NEFT/RTGS).
2. **Fraud check.** Is the payee on the **watchlist**? Is the amount **huge** (>= Rs 10,00,000)? Is the **live session risk high** (>= 70, i.e. the insider engine already distrusts this session)? If any is true -> **FLAGGED**: the money does **not** move and a **CRITICAL** alert flashes to the SOC.
3. **High-value check.** If it's large (> Rs 2,00,000) -> **HELD**: parked for a **second officer** to approve or reject (maker-checker). Nothing moves yet.
4. **Otherwise -> CLEARED:** the money moves immediately (assuming funds are there and the account isn't frozen).

Why it matters: it shows the insider engine and the banking floor are **one loop** — the same risk score that flags a session also **stops the money**. The judges see fraud controls acting on *real transactions*, not a toy demo.

PART 4 — Every word explained simply

- **Privileged user** — a staff member with powerful access (admin, DBA, IT, vendor). *The person with the master key.*
- **Insider threat** — danger from someone already inside with access, not an outside hacker.
- **PAM (Privileged Access Management)** — the discipline of controlling, recording, and reviewing what privileged users do. *CCTV + rules for the master-key holders.*
- **SOC (Security Operations Centre)** — the bank's security control room; **analysts** watch and respond.
- **Session** — one login-to-logout period for a user.
- **Event** — one action inside a session (query, file open, export...).

- **Rule engine** — a fixed checklist of known-bad behaviours; each match adds "weight."
- **UEBA (User & Entity Behaviour Analytics)** — AI that learns each user's *normal* behaviour and flags the *unusual*. *A guard who knows everyone's habits.*
- **IsolationForest** — the specific AI algorithm we use; it's good at spotting "odd ones out" **without needing labelled examples of attacks** (unsupervised). *It notices the item that doesn't fit.*
- **Per-user baseline** — each person's own usual hours, devices, locations, and data volumes, so we can say "this is unusual **for this person**."
- **Feature vector** — the seven numbers describing a session (login hour, action count, records touched, distinct systems, config changes, off-network?, new device?) that the AI reads.
- **Risk score (0–100)** — the single number combining rules + AI + peer comparison.
- **Insider type** — malicious / negligent / compromised — decided by which rule family carries the most weight.
- **Response ladder** — allow -> step-up MFA -> maker-checker -> block, chosen by the score.
- **Step-up MFA** — asking for an extra one-time code to continue a medium-risk action. *"Prove it's really you."*
- **Maker-checker** — a high-risk action is **held** until a *second* authorised person approves it. *"Two keys to launch."*
- **Block** — refuse the action and lock the account.
- **WebSocket** — a live, always-open connection that lets the server **push** updates to the screen instantly (no refresh). *A phone line left open.*
- **Post-quantum cryptography (PQC)** — new encryption that stays safe even against future quantum computers.
- **ML-KEM-768** — the NIST post-quantum *key-agreement* algorithm (FIPS 203); used to seal the vault's keys.
- **ML-DSA-65** — the NIST post-quantum *digital-signature* algorithm (FIPS 204); used to sign each audit entry.
- **AES-256-GCM** — strong, standard symmetric encryption used to lock the actual secret (the key for it comes from ML-KEM).
- **Hash / hash chain** — a "fingerprint" of data; a chain where each entry embeds the previous entry's fingerprint, so editing one breaks all the following ones. *Blockchain-style tamper evidence.*
- **Audit log** — the permanent record of everything that happened.
- **Harvest now, decrypt later** — a criminal steals encrypted data today to decrypt it years later with a quantum computer; PQC defeats this.
- **PBKDF2** — a safe way to store passwords (slow, salted hashing) so they can't be reversed.
- **Token** — a signed pass the browser shows on each request to prove who is logged in.
- **FastAPI / Uvicorn** — the Python web-server tools serving the API.
- **SQLite / PostgreSQL** — the database; SQLite for the offline demo, Postgres for scale.
- **SIEM** — a bank's central log-collection system; a real source of the events we score.
- **Core-banking desk** — the working Accounts / Payments / Transactions / Approvals screens in the Employee Portal, backed by a real ledger.
- **Maker-checker (banking)** — a large transfer is **held** until a *second* officer approves it; the maker cannot approve their own.
- **Flagged / fraud** — a transfer to a watchlisted payee, a huge amount, or one from a high-risk session; the money is stopped and a CRITICAL alert flashes to the SOC.
- **AI Model Insights** — the SOC panel that shows *which* behaviours (features) moved the score, each versus the user's own baseline and their peers — the explainable face of the AI.
- **Risk trajectory** — the little sparkline of how the score climbed action by action, showing where it crossed a response threshold.
- **Impact metrics** — the live strip of program value: sessions watched, threats blocked, money held/flagged, mean time to decision.
- **PQC provider fallback** — if the native crypto library isn't installed, Prahari uses a **pure-Python** one automatically; no compiler, so it runs on any machine.

PART 5 — Cheat-sheet (memorise these)

One-liner: *Prahari catches risky insiders — malicious, negligent, or compromised — in real time, responds automatically, and keeps the proof quantum-safe.*

Run it: `.\run.ps1 -Reset` -> open `http://localhost:8000`. Second computer on the same Wi-Fi uses the LAN URL the script prints.

Demo accounts (password `prahari123`):

Login	Who	Use for
`soc_admin`	SOC analyst	the **SOC Console**
`rmehta`	normal DBA	a normal employee
`ext_dsouza`	dormant vendor	the **malicious** attacker
`ext_rao`	expired vendor	the **negligent** case

Step-up MFA code: `246810`.

The three scenarios and their outcomes:

Type	Story	Score	Response
Malicious	dormant vendor, 2 AM, export 5,000 records	100	**BLOCK**
Compromised	new country + device, rapid-fire	~54	**STEP-UP MFA**
Negligent	expired vendor, personal laptop	~63	**MAKER-CHECKER**

Rule weights (roughly): dormant reactivation 30 · privilege escalation 25 · after-hours 20 · mass export (>=1000) 30 · out-of-role 15 · new-geo 16 · new-device 12 · atypical-hour 8 · rapid-fire 8 · expired-access 30 · unmanaged-device 30.

Score formula: `min(rule points (cap 80) + 0.25 × AI anomaly + 10 peer bonus, 100)`.

Banking demo: on the portal's **Payments** tab — a small transfer **clears**, one over Rs 2,00,000 is **HELD** for a second officer, and one to a watchlisted payee is **FLAGGED** and flashes a red banner + a CRITICAL SOC alert.

Crypto: vault = **ML-KEM-768 + AES-256-GCM**; audit = **ML-DSA-65** signatures over a **hash chain**. **No compiler needed** — native library or a pure-Python fallback (`GET /pqc/info` shows which).

Proof it works: 40 automated tests pass; the three scenarios land on **three different** responses; a flagged transfer stops real money; **tamper** breaks the audit chain instantly; runs **fully offline** with **no build step**; works **live across two computers**.

Six judged outcomes — all delivered: detect misuse · real-time · AI behavioural · risk-based access control · protect admin systems · quantum-proof crypto.

Prahari — the sentinel for privileged access.